



Exercice 1

Écrire une fonction `disBonjour(n)` qui affiche le mot `bonjour` n fois.

Exemple : `disBonjour(3)` affiche :

```
bonjour
bonjour
bonjour
```

- Écrire et tester une fonction `estEntre(val, a, b)` qui retourne `True` si `val` est compris entre `a` et `b`, sinon `False`.
- Écrire une fonction `conversion` qui reçoit une durée en heures et la convertit en minutes et secondes.

Exercice 2

- Écrire une fonction `nbMots` qui retourne le nombre de mots dans une chaîne de caractères.
Exemple : `nbMots('Bonjour tout le monde !')` $\Rightarrow$ 4.
- Écrire une fonction qui retourne le nombre de voyelles contenues dans une chaîne.
- Écrire une fonction qui vérifie si une chaîne est dite *carrée*, c'est-à-dire de la forme `c = uu`.

Exercice 3

- Écrire une fonction `est_palindrome(mot)` qui retourne `True` si le mot est un palindrome (sans tenir compte de la casse), sinon `False`.
- Écrire un programme qui demande un mot à l'utilisateur et affiche s'il s'agit ou non d'un palindrome.

Exercice 4

- Une liste $l = [x_1, x_2, x_3, \dots, x_n]$ est dite *alternée* si :
 - Chaque élément est de signe opposé à celui qui le précède.
 - Les indices pairs sont égaux à eux-mêmes.
 - Les indices impairs sont égaux à leur opposé.

Exemple : `[0, -1, 2, -3, 4, -5]` est alternée.

Écrire une fonction `alternee(1)` qui vérifie cela.

Exercice 5

- Écrire une fonction récursive `egal_recurs(L, M)` qui retourne `True` si les deux listes `L` et `M` sont égales élément par élément.
- Écrire un programme qui trie une liste `L` en ordre décroissant.

Exercice 6

- Écrire une fonction `elimine(L)` qui supprime les doublons dans une liste.
Exemple : `elimine([1,2,2,1,2])` $\Rightarrow$ `[1,2]`.
- Proposer une version récursive de la fonction.



F7

La suite de Fibonacci est définie par : $f_0 = f_1 = 1$, et pour $n > 1$, $f_n = f_{n-1} + f_{n-2}$.

Écrire une fonction `f(n)` qui retourne le n -ième terme de la suite.

- Proposer une version récursive de la fonction.

Exercice 8

- Écrire une fonction récursive `sommeR(n)` qui calcule la somme suivante :

$$S = 1 + 3^2 + 5^4 + 7^6 + \dots + (2n + 1)^{2n}$$

- Écrire une fonction `trouver_k(n)` qui renvoie l'unique entier $k \geq 0$ tel que :

$$2^k \leq n < 2^{k+1}$$

- Proposer une version récursive de la fonction `trouver_k(n)`.

Exercice 9

- On suppose que vous gérez un stock de produits dans un magasin. Chaque produit est représenté par un dictionnaire où la clé est le nom du produit et la valeur est la quantité disponible en stock. Créer un dictionnaire contenant au moins 4 produits avec leurs quantités initiales. Par exemple :

```
stock = {
    "Pomme": 50,
    "Banane": 30,
    "Orange": 20,
    "Poire": 15
}
```

- Écrire une fonction `afficher_stock(stock)` qui affiche les produits et leurs quantités. Exemple d'affichage :

```
Pomme: 50
Banane: 30
Orange: 20
Poire: 15
```

- Écrire une fonction `ajouter_produit(stock, produit, quantite)` qui ajoute un produit au stock ou augmente sa quantité si le produit existe déjà. Exemple :

```
ajouter_produit(stock, "Pomme", 10)
```

Après cela, la quantité de "Pomme" doit être de 60.

- Écrire une fonction `retirer_produit(stock, produit, quantite)` qui enlève une quantité donnée du stock si le stock est suffisant. Si le produit n'est pas en stock ou que la quantité demandée est trop élevée, afficher un message d'erreur. Exemple :



`irer_produit(stock, "Banane", 5)`

Cela, la quantité de "Banane" doit être de 25.

- Écrire une fonction `vendre_produit(stock, produit, quantite)` qui simule la vente d'un produit. La fonction doit vérifier si la quantité en stock est suffisante avant d'effectuer la vente et retourner le message : "Vente réussie" ou "Stock insuffisant". Exemple :

`vendre_produit(stock, "Orange", 25)`

- Écrire une fonction `inventaire_total(stock)` qui calcule la quantité totale de produits en stock. Exemple :

`inventaire_total(stock)`

Cela doit retourner la somme de toutes les quantités présentes dans le stock.

- Ajouter une fonctionnalité pour gérer les produits avec des prix. Le dictionnaire de chaque produit doit être modifié pour inclure un prix. Par exemple :

```
stock = {
    "Pomme": {"quantite": 50, "prix": 1.5},
    "Banane": {"quantite": 30, "prix": 1.2},
    "Orange": {"quantite": 20, "prix": 1.8},
    "Poire": {"quantite": 15, "prix": 2.0}
}
```

Créer une fonction `calculer_valeur_stock(stock)` qui calcule la valeur totale du stock en multipliant la quantité de chaque produit par son prix.

- Écrire une fonction `afficher_produits_disponibles(stock)` qui affiche tous les produits dont la quantité est supérieure à zéro. Exemple :

`afficher_produits_disponibles(stock)`

Cela devrait afficher uniquement les produits ayant des quantités positives.

- Ajouter une fonction `rechercher_produit(stock, produit)` qui recherche un produit dans le stock et retourne ses informations (quantité et prix). Si le produit n'existe pas, afficher un message indiquant qu'il est introuvable.
- Créer une fonction `vider_stock(stock)` qui vide le stock en mettant la quantité de chaque produit à zéro.